UNIT 5 – GRAPHS

Topic: Modified Graph Operations Using BFS and DFS

1. Problem Statement

Modify the original menu-driven C++ graph program to accept additional vertices and edges and display the updated graph. Compare BFS and DFS traversal results before and after graph updates. The program should also handle disconnected vertices and display an appropriate message when a path between two vertices is unavailable.

2. Objectives

1. To add additional vertices and edges to a graph.
2. To display the graph after every update.
3. To implement BFS and DFS traversal.
4. To identify disconnected vertices.
5. To check whether a path exists between two vertices.
6. To compare graph information before and after updates.

3. Theory

A graph consists of vertices and edges. Vertices represent objects, while edges represent connections between them.

Adjacency List

An adjacency list stores the neighbouring vertices of each vertex. It is useful for representing graph connections.

Breadth-First Search (BFS)

BFS visits neighbouring vertices first. It uses a queue to process vertices in traversal order.

Depth-First Search (DFS)

DFS explores one path deeply before returning to explore other paths. It can be implemented using recursion.

Graph Connectivity

Two vertices are connected if a path exists between them. If no path exists, they are considered disconnected.

4. Modifications Implemented

Modification A: Additional Vertices and Edges

The program allows users to insert additional vertices and edges. The updated adjacency list can be displayed after each graph update.

Modification B: Traversal Comparison

The program displays BFS and DFS traversal results for the current graph. These results can be compared with the results recorded before an update.

Note: The current code does not automatically store and compare complete traversal sequences from before and after an update.

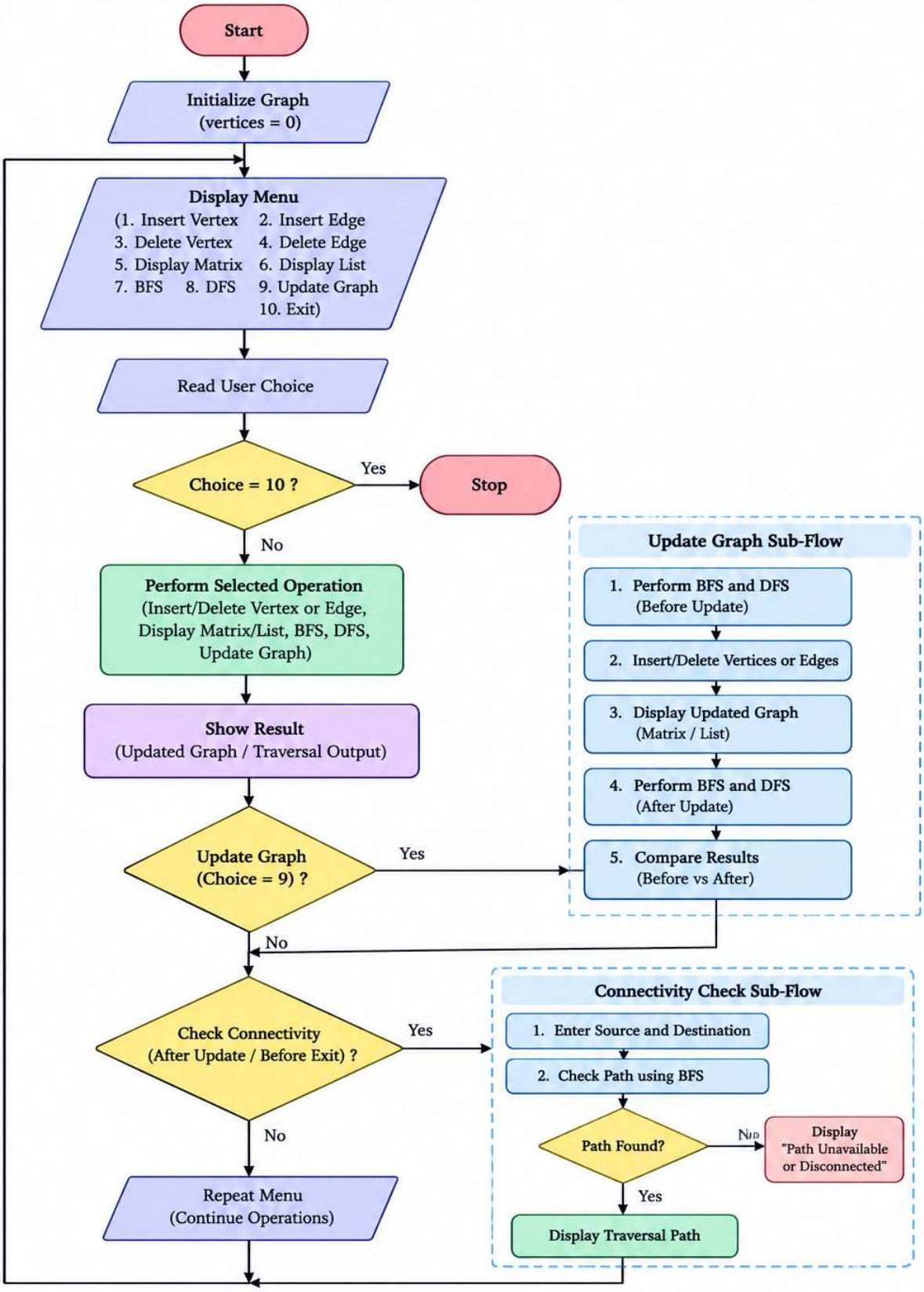
Modification C: Disconnected Vertices and Unavailable Paths

The program identifies vertices that cannot be reached from the selected starting vertex during BFS and DFS. It also checks whether a path exists between two selected vertices and displays an appropriate message.

5. Algorithm

1. Start the program.
2. Initialize an empty graph.
3. Display the menu.
4. Accept the user's choice.
5. Perform the selected graph operation.
6. Display the adjacency matrix or adjacency list when requested.
7. Perform BFS or DFS from the selected starting vertex.
8. Identify vertices that are not reachable from the starting vertex.
9. Check connectivity between the selected source and destination.
10. Display whether the path is available or unavailable.
11. Update the graph by inserting or deleting vertices or edges.
12. Display the updated graph.
13. Repeat the menu until the user selects Exit.
14. Stop the program.

6. Flowchart



7. Source Code

```
#include <iostream>
#include <vector>
#include <queue>
using namespace std;
```

```
vector<int> adj[100];
int vertices = 0;
void insertVertex() {
    if (vertices >= 100) {
        cout << "Maximum vertices reached.\n";
        return;
    }
    vertices++;
    cout << "Vertex " << vertices - 1 << " added.\n";
}
void insertEdge() {
    int u, v, i;
    cout << "Enter two vertex numbers: ";
    cin >> u >> v;
    if (u < 0 || v < 0 || u >= vertices || v >= vertices) {
        cout << "Invalid vertex number.\n";
        return;
    }
    for (i = 0; i < (int)adj[u].size(); i++) {
        if (adj[u][i] == v) {
            cout << "Edge already exists.\n";
            return;
        }
    }
    adj[u].push_back(v);
    adj[v].push_back(u);
    cout << "Edge added successfully.\n";
}
void deleteEdge() {
    int u, v, i;
    cout << "Enter edge vertices to delete: ";
    cin >> u >> v;
    if (u < 0 || v < 0 || u >= vertices || v >= vertices) {
        cout << "Invalid vertex number.\n";
        return;
    }
```

```
    for (i = 0; i < (int)adj[u].size(); i++) {
        if (adj[u][i] == v) {
            adj[u].erase(adj[u].begin() + i);
            break;
        }
    }
    for (i = 0; i < (int)adj[v].size(); i++) {
        if (adj[v][i] == u) {
            adj[v].erase(adj[v].begin() + i);
            break;
        }
    }
    cout << "Edge deletion completed.\n";
}
void deleteVertex() {
    int v, i, j;
    cout << "Enter vertex number to delete: ";
    cin >> v;
    if (v < 0 || v >= vertices) {
        cout << "Invalid vertex number.\n";
        return;
    }
    for (i = 0; i < vertices; i++) {
        for (j = 0; j < (int)adj[i].size(); j++) {
            if (adj[i][j] == v) {
                adj[i].erase(adj[i].begin() + j);
                j--;
            } else if (adj[i][j] > v) {
                adj[i][j]--;
            }
        }
    }
    for (i = v; i < vertices - 1; i++) {
        adj[i] = adj[i + 1];
    }
    adj[vertices - 1].clear();
```

```
    vertices--;

    cout << "Vertex deleted successfully.\n";
}

void displayMatrix() {
    int i, j, k, found;

    cout << "\nAdjacency Matrix:\n";

    for (i = 0; i < vertices; i++) {
        for (j = 0; j < vertices; j++) {
            found = 0;
            for (k = 0; k < (int)adj[i].size(); k++) {
                if (adj[i][k] == j) {
                    found = 1;
                    break;
                }
            }
            cout << found << " ";
        }
        cout << endl;
    }
}
void displayList() {
    int i, j;
    cout << "\nAdjacency List:\n";
    for (i = 0; i < vertices; i++) {
        cout << i << " -> ";
        for (j = 0; j < (int)adj[i].size(); j++) {
            cout << adj[i][j] << " ";
        }
        cout << endl;
    }
}
```

```
void traverseBFS(int start, vector<bool>& visited) {
    queue<int> q;
    int v, i, x;
    visited[start] = true;
    q.push(start);
    while (!q.empty()) {
        v = q.front();
        q.pop();
        cout << v << " ";

        for (i = 0; i < (int)adj[v].size(); i++) {
            x = adj[v][i];

            if (!visited[x]) {
                visited[x] = true;
                q.push(x);
            }
        }
    }
}
void BFS() {
    int start, i;

    if (vertices == 0) {
        cout << "Graph is empty.\n";
        return;
    }
    cout << "Enter starting vertex: ";
    cin >> start;
    if (start < 0 || start >= vertices) {
        cout << "Invalid starting vertex.\n";
        return;
    }
    vector<bool> visited(vertices, false);
    cout << "BFS Traversal: ";
    traverseBFS(start, visited);
```

```
    cout << endl;
    for (i = 0; i < vertices; i++) {
        if (!visited[i]) {
            cout << "Vertex " << i << " is disconnected from "
                << start << ".\n";
        }
    }
}
void DFSUtil(int v, vector<bool>& visited) {
    int i, x;

    visited[v] = true;
    cout << v << " ";
    for (i = 0; i < (int)adj[v].size(); i++) {
        x = adj[v][i];

        if (!visited[x]) {
            DFSUtil(x, visited);
        }
    }
}
void DFS() {
    int start, i;

    if (vertices == 0) {
        cout << "Graph is empty.\n";
        return;
    }
    cout << "Enter starting vertex: ";
    cin >> start;

    if (start < 0 || start >= vertices) {
        cout << "Invalid starting vertex.\n";
        return;
    }
    vector<bool> visited(vertices, false);
```

```
    cout << "DFS Traversal: ";
    DFSUtil(start, visited);
    cout << endl;

    for (i = 0; i < vertices; i++) {
        if (!visited[i]) {
            cout << "Vertex " << i << " is disconnected from "
                 << start << ".\n";
        }
    }
}
bool isReachable(int start, int target) {
    vector<bool> visited(vertices, false);
    queue<int> q;
    int v, i, x;
    visited[start] = true;
    q.push(start);

    while (!q.empty()) {
        v = q.front();
        q.pop();

        if (v == target)
            return true;
        for (i = 0; i < (int)adj[v].size(); i++) {
            x = adj[v][i];

            if (!visited[x]) {
                visited[x] = true;
                q.push(x);
            }
        }
    }
    return false;
}
```

```
void checkPath() {
    int source, destination;

    if (vertices == 0) {
        cout << "Graph is empty.\n";
        return;
    }

    cout << "Enter source and destination: ";
    cin >> source >> destination;

    if (source < 0 || destination < 0 ||
        source >= vertices || destination >= vertices) {
        cout << "Invalid vertex number.\n";
        return;
    }

    if (isReachable(source, destination))
        cout << "Path is available.\n";
    else
        cout << "Path unavailable: vertices are disconnected.\n";
}

void compareTraversal() {
    int start;

    if (vertices == 0) {
        cout << "Graph is empty.\n";
        return;
    }
    cout << "Enter starting vertex: ";
    cin >> start;
    if (start < 0 || start >= vertices) {
        cout << "Invalid starting vertex.\n";
        return;
    }
```

```
    vector<bool> visited(vertices, false);

    cout << "\nBFS before/after update comparison:\n";
    cout << "Current BFS traversal: ";
    traverseBFS(start, visited);
    cout << "\n";

    for (int i = 0; i < vertices; i++)
        visited[i] = false;

    cout << "Current DFS traversal: ";
    DFSUtil(start, visited);
    cout << "\n";
    cout << "Compare this result with the traversal recorded before updating.\n";
}

void updateGraph() {
    int choice;
    vector<vector<int> > oldGraph(vertices);
    for (int i = 0; i < vertices; i++)
        oldGraph[i] = adj[i];
    int oldVertices = vertices;
    cout << "\n1. Insert Vertex\n";
    cout << "2. Insert Edge\n";
    cout << "3. Delete Vertex\n";
    cout << "4. Delete Edge\n";
    cout << "Enter update choice: ";
    cin >> choice;

    switch (choice) {
        case 1: insertVertex(); break;
        case 2: insertEdge(); break;
        case 3: deleteVertex(); break;
        case 4: deleteEdge(); break;
        default:
```

```
            cout << "Invalid choice.\n";
            return;
        }
        cout << "\nGraph after update:\n";
        displayList();

        cout << "\nPrevious graph had " << oldVertices
             << " vertices; updated graph has " << vertices << " vertices.\n";

        cout << "Traversal comparison: run BFS/DFS before and after the update "
             << "to compare the results.\n";
    }
    int main() {
        int choice;

        do {
            cout << "\n===== MODIFIED GRAPH OPERATIONS =====\n";
            cout << "1. Insert Vertex\n";
            cout << "2. Insert Edge\n";
            cout << "3. Delete Vertex\n";
            cout << "4. Delete Edge\n";
            cout << "5. Display Adjacency Matrix\n";
            cout << "6. Display Adjacency List\n";
            cout << "7. BFS Traversal\n";
            cout << "8. DFS Traversal\n";
            cout << "9. Update Graph\n";
            cout << "10. Check Path Connectivity\n";
            cout << "11. Compare Traversal\n";
            cout << "12. Exit\n";
            cout << "Enter choice: ";
            cin >> choice;
            switch (choice) {
                case 1: insertVertex(); break;
                case 2: insertEdge(); break;
                case 3: deleteVertex(); break;
                case 4: deleteEdge(); break;
```

```
            case 5: displayMatrix(); break;
            case 6: displayList(); break;
            case 7: BFS(); break;
            case 8: DFS(); break;
            case 9: updateGraph(); break;
            case 10: checkPath(); break;
            case 11: compareTraversal(); break;
            case 12: cout << "Program ended.\n"; break;
            default: cout << "Invalid choice.\n";
        }
    } while (choice != 12);
    return 0;
}
```

8. Sample Output



```
===== MODIFIED GRAPH OPERATIONS =====
1. Insert Vertex
2. Insert Edge
3. Delete Vertex
4. Delete Edge
5. Display Adjacency Matrix
6. Display Adjacency List
7. BFS Traversal
8. DFS Traversal
9. Update Graph
10. Check Path Connectivity
11. Compare Traversal
12. Exit
Enter choice: 6

Adjacency List:
0 -> 1
1 -> 0 2
2 -> 1
3 ->

===== MODIFIED GRAPH OPERATIONS =====
1. Insert Vertex
2. Insert Edge
3. Delete Vertex
4. Delete Edge
5. Display Adjacency Matrix
6. Display Adjacency List
7. BFS Traversal
8. DFS Traversal
9. Update Graph
10. Check Path Connectivity
11. Compare Traversal
12. Exit
Enter choice: 7
Enter starting vertex: 8
Invalid starting vertex.

===== MODIFIED GRAPH OPERATIONS =====
1. Insert Vertex
2. Insert Edge
3. Delete Vertex
4. Delete Edge
5. Display Adjacency Matrix
6. Display Adjacency List
7. BFS Traversal
8. DFS Traversal
9. Update Graph
10. Check Path Connectivity
11. Compare Traversal
12. Exit
Enter choice: 10
Enter source and destination: 0 3
Path unavailable: vertices are disconnected.
```

```
===== MODIFIED GRAPH OPERATIONS =====
1. Insert Vertex
2. Insert Edge
3. Delete Vertex
4. Delete Edge
5. Display Adjacency Matrix
6. Display Adjacency List
7. BFS Traversal
8. DFS Traversal
9. Update Graph
10. Check Path Connectivity
11. Compare Traversal
12. Exit
Enter choice: 9

1. Insert Vertex
2. Insert Edge
3. Delete Vertex
4. Delete Edge
Enter update choice: 2
Enter two vertex numbers: 2 3
Edge added successfully.

Graph after update:

Adjacency List:
0 -> 1
1 -> 0 2
2 -> 1 3
3 -> 2

Previous graph had 4 vertices; updated graph has 4 vertices.
Traversal comparison: run BFS/DFS before and after the update to compare the results.

===== MODIFIED GRAPH OPERATIONS =====
1. Insert Vertex
2. Insert Edge
3. Delete Vertex
4. Delete Edge
5. Display Adjacency Matrix
6. Display Adjacency List
7. BFS Traversal
8. DFS Traversal
9. Update Graph
10. Check Path Connectivity
11. Compare Traversal
12. Exit
Enter choice: 10
Enter source and destination: 0 3
Path is available.
```

9. Test Cases

Test Case	Input / Operation	Expected Result
1	Insert a vertex	New vertex is added
2	Insert an edge	Connection is added
3	Display adjacency list	Neighbouring vertices are displayed
4	Perform BFS	Reachable vertices are traversed
5	Perform DFS	Reachable vertices are traversed
6	Check path between connected vertices	Path is available
7	Check path between disconnected vertices	Path unavailable message
8	Add an edge between disconnected components	Updated graph shows the new connection

10. Applications

1. Campus navigation systems.
2. Computer network connectivity.
3. Social networking applications.
4. Route and path exploration.
5. Network structure analysis.

11. Result

The modified graph program supports graph updates, adjacency matrix and adjacency list display, BFS and DFS traversal, disconnected vertex detection, and path connectivity checking.

12. Conclusion

This practical improved the understanding of graph representation, traversal, and connectivity. It demonstrated how graph updates affect connections between vertices and how BFS can be used to check whether a path exists between two vertices.